

Proiect SI

- Masurarea distantei cu senzor capacitiv -

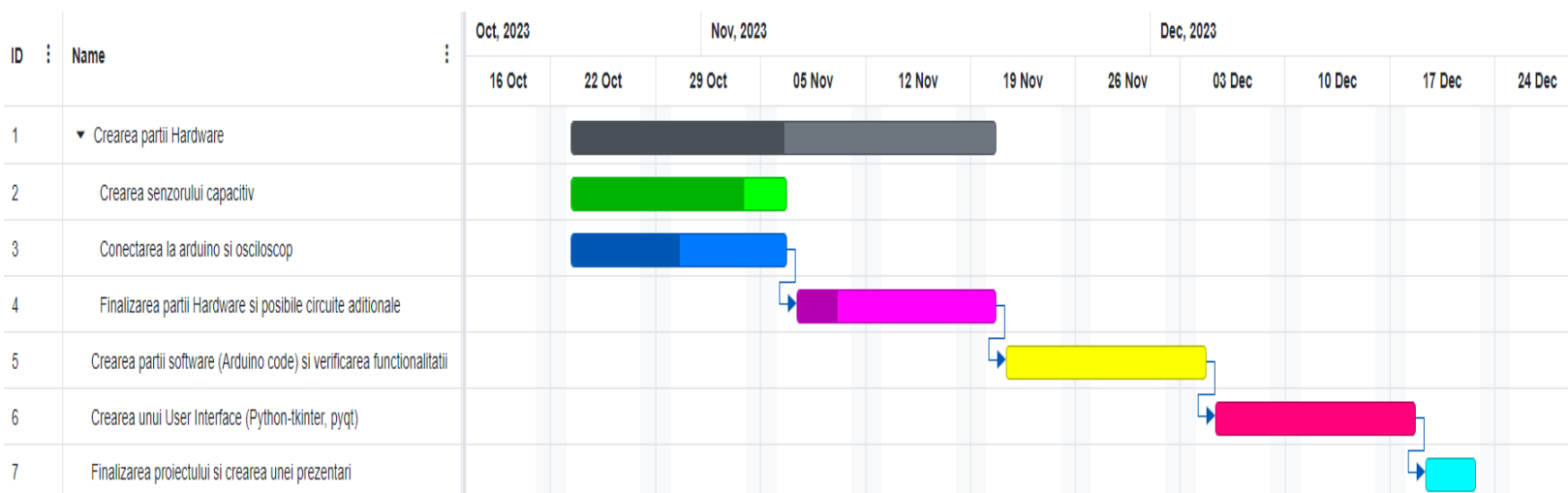
Nume: Condrea Mihail-Alexandru

Diaconescu Serban-Cristian

Burlacu Gabriel

Grupa: 6403

Obiective	Perioada
Construirea senzorului capacitiv si verificarea limitelor acestuia. Inceperea conectarii acestuia cu Arduino si osciloscop.	23OCT-6NOV
Finalizarea partii hardware, de conectare a senzorului si posibile circuite aditionale.	7NOV-20NOV
Crearea partii software a proiectului si verificarea functionalitatii.	21NOV-04DEC
Crearea unui user interface Serial-computer in python folosind libraria tkinter sau pyqt	05DEC-18DEC
Finalizarea proiectului si crearea unei prezentari a acestuia. (documentatie mai detaliata)	19DEC-22DEC



Notiuni de baza senzor capacitiv si distanta

- **Distanta reprezinta separarea spatiala dintre doua puncte.**

Senzorii capacitivi sunt senzori de tip pasiv care convertesc o marime neelectrică de masurat – deplasare liniară - într-o variație de capacitate electrică.

$$C = \frac{\epsilon \cdot S}{\delta} \quad \text{sau} : C = \frac{\epsilon \cdot S}{\delta \pm x}$$

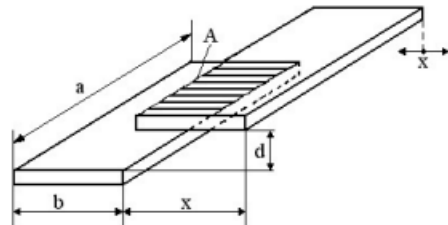
Un senzor capacitiv este un tip de senzor care detectează și măsoară schimbările în capacitățile electrice ale unui mediu sau al unui material. Funcționează pe principiul capacității electrice, care este proprietatea unui sistem electric de a stoca o sarcină electrică.

Senzorii capacitivi sunt utilizați într-o varietate de aplicații, de la ecrane tactile și taste capacitivă, la măsurarea nivelului de lichide și a obiectelor în procesele industriale.

Un avantaj al senzorilor capacitivi este că sunt sensibili și pot detecta schimbări foarte mici în capacități, făcându-i utili în aplicații de precizie.

- **Elemente sensibile capacitive cu modificarea suprafeței active dintre armături**

Se consideră armăturile unui condensator plan cu dimensiunile a , b și se notează cu x deplasarea relativă a unei armături față de cealaltă ca în figura de mai jos:



Dependența capacității de deplasarea x la acest element sensibil este o aproximativ liniară:

$$\Delta C = \frac{-\varepsilon \cdot a \cdot x}{d}$$

Iar sensibilitatea acestui tip de element sensibil se poate exprima prin relația:

$$S = \frac{\Delta C}{\Delta x} = -\frac{\varepsilon \cdot A}{d}$$

Adică pentru ε și d constanți, sensibilitatea (S) va fi cu atât mai mare, cu cât suprafața activă (A) dintre armături crește, iar distanța (d) dintre acestea se micșorează.

Elementele sensibile capacitive cu modificarea suprafeței active dintre armături se referă la o componentă specifică a senzorilor capacitivi, și anume la partea efectivă care detectează schimbările de capacitate. Aceste elemente sensibile sunt esențiale în designul și funcționarea senzorilor capacitivi.

Modificarea Suprafeței Active: Aceasta se referă la schimbarea efectivei zone de interacțiune între armături. De exemplu, dacă un obiect metalic se apropie de senzor, suprafața activă (zona în care câmpul electric este semnificativ) se poate schimba.

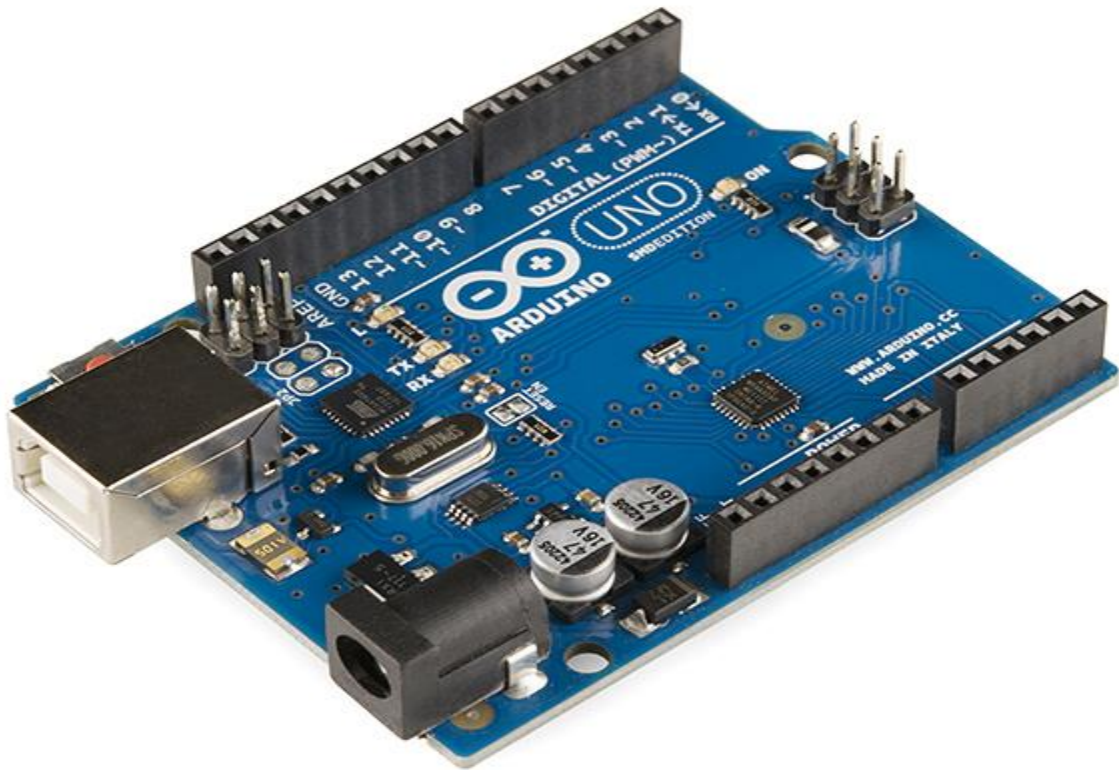
Elementele sensibile sunt proiectate pentru a fi foarte receptive la aceste tipuri de schimbări, permițând senzorilor capacitivi să detecteze chiar și variații minore. Astfel de senzori sunt larg răspândiți în diverse domenii, inclusiv în industrie, sisteme de control automat, dispozitive electronice de consum și multe altele. Aceste elemente permit senzorilor să ofere măsurători precise și fiabile, făcându-i ideali pentru aplicații unde sensibilitatea și acuratețea sunt critice.

Proiectul este formati din doua parti: o parte hardware(1) si o parte software(2).

(1)Partea Hardware

Componentele proiectului:

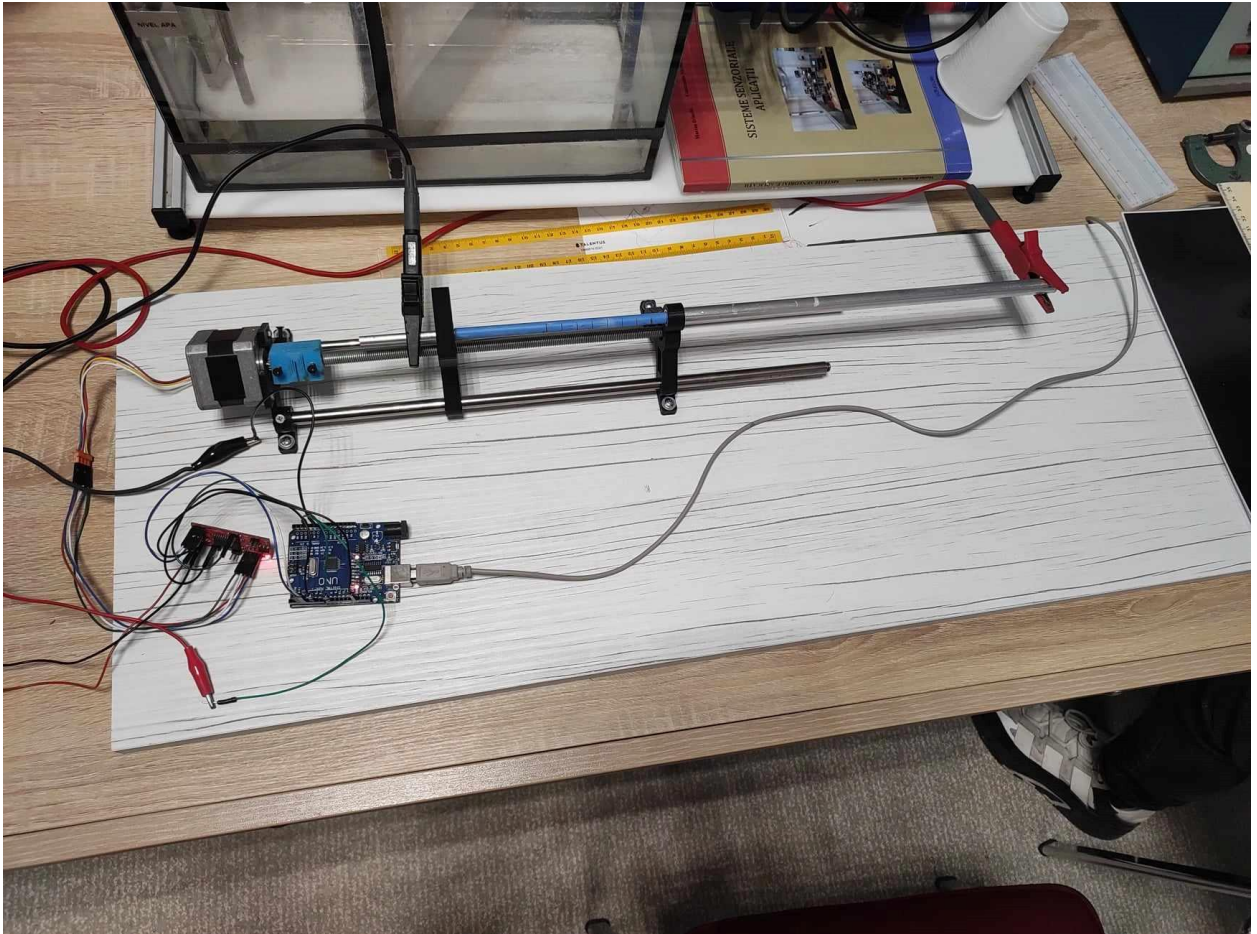
❖ **Arduino uno**

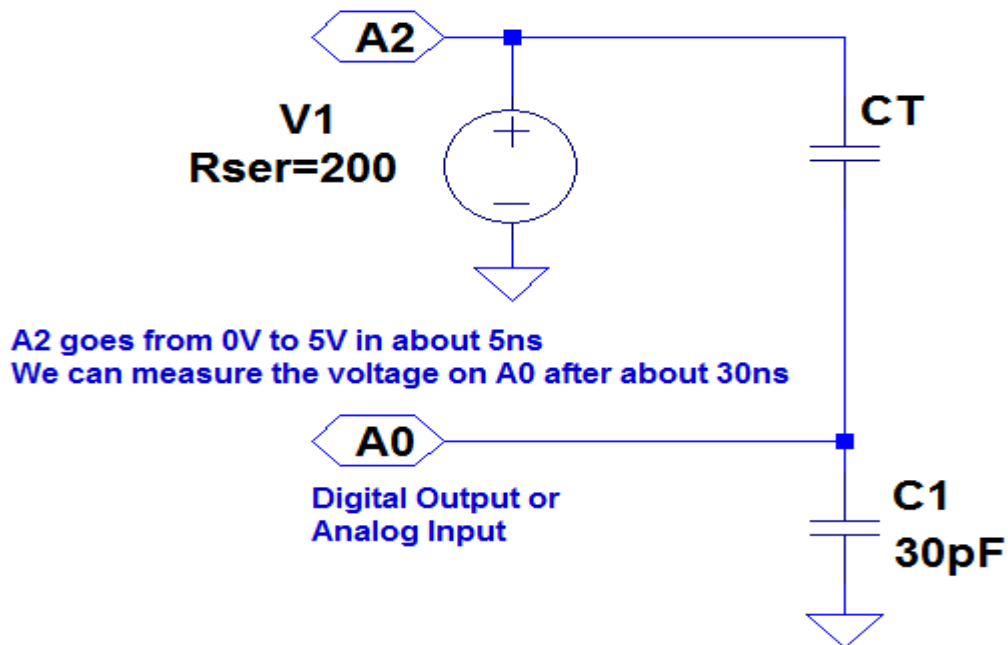


Atribuire Share-Alike 2.5 și este disponibil pe site-ul Arduino. Fișierele de layout și producție pentru unele versiuni ale hardware-ului sunt, de asemenea, disponibile.

❖ **Senzor capacitiv**

Senzorul capacitiv, eficient și simplu, este alcătuit din două plăci metalice cilindrice, conectate la un Arduino UNO. Aceste plăci sunt legate la doi pini analogici pe Arduino (A2 și A0). Modificând distanța sau poziția între cele două plăci metalice, se schimbă capacitatea dintre ele. Această schimbare este detectabilă și măsurabilă cu ajutorul Arduino, oferind o metodă directă și practică pentru a monitoriza variațiile de capacitate.





C_T este condensatorul testat. Începem cu ambele condensatoare descărcate și A2 la 0 volți. Când ridicăm A2 la 5 volți, un curent va curge prin ambele condensatoare. Tensiunea pe A0 se va stabili la aproximativ 1% din valoarea sa finală în 30ns. Valoarea la care se va stabili A0 este proporțională cu raportul dintre C_T și capacitanța totală, $C_1 + C_T$. Formulele de care vom avea nevoie sunt:

$$V_{A0} = \frac{V_{A2} \times C_T}{C_1 + C_T}$$

We can rearrange this to get:

$$C_1 = \frac{C_T \times (V_{A2} - V_{A0})}{V_{A0}}$$

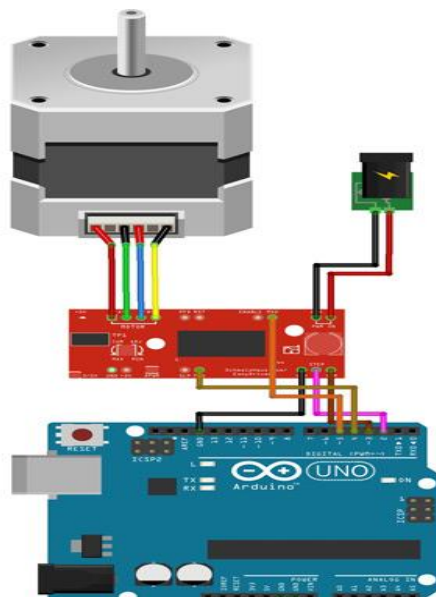
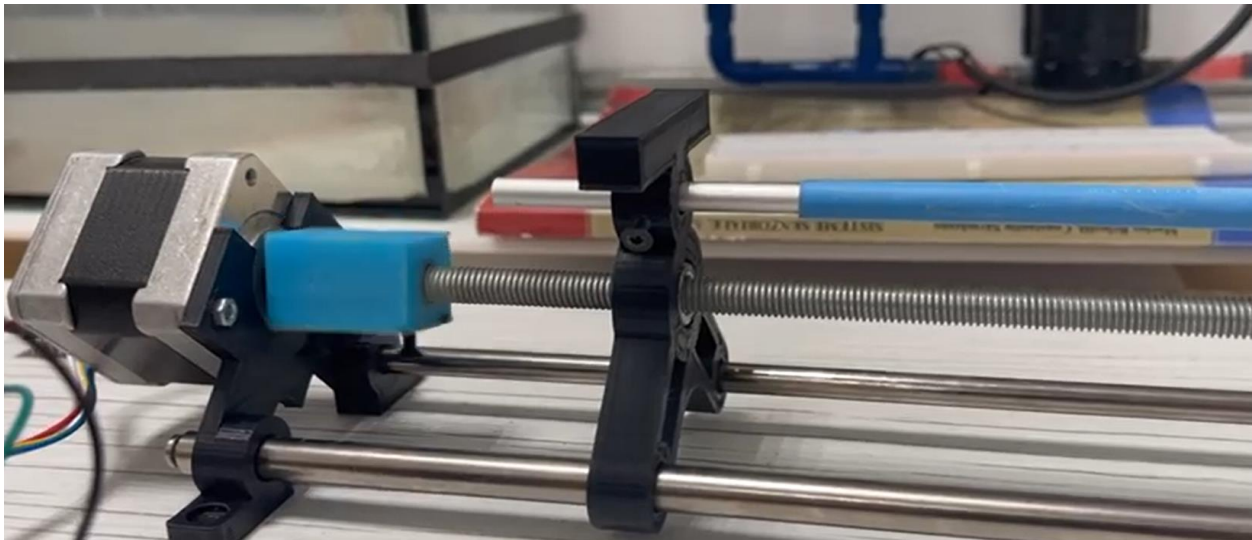
and

$$C_T = \frac{V_{A0} \times C_1}{V_{A2} - V_{A0}}$$

Experimente SENZOR CAPACITIV (atat cu Arduino UNO cat si Arduino MEGA) :

Punte(pF)	Arduino Uno(pF)	Arduino Mega(Pf)
104.8	100.588	96.147
85.2	81.182	78.013
175	166.138	158.74
147	140.745	135.154
1034	1023.75	978.261

❖ Motor pas cu pas si driver (EasyDriver + Sursa de tensiune)

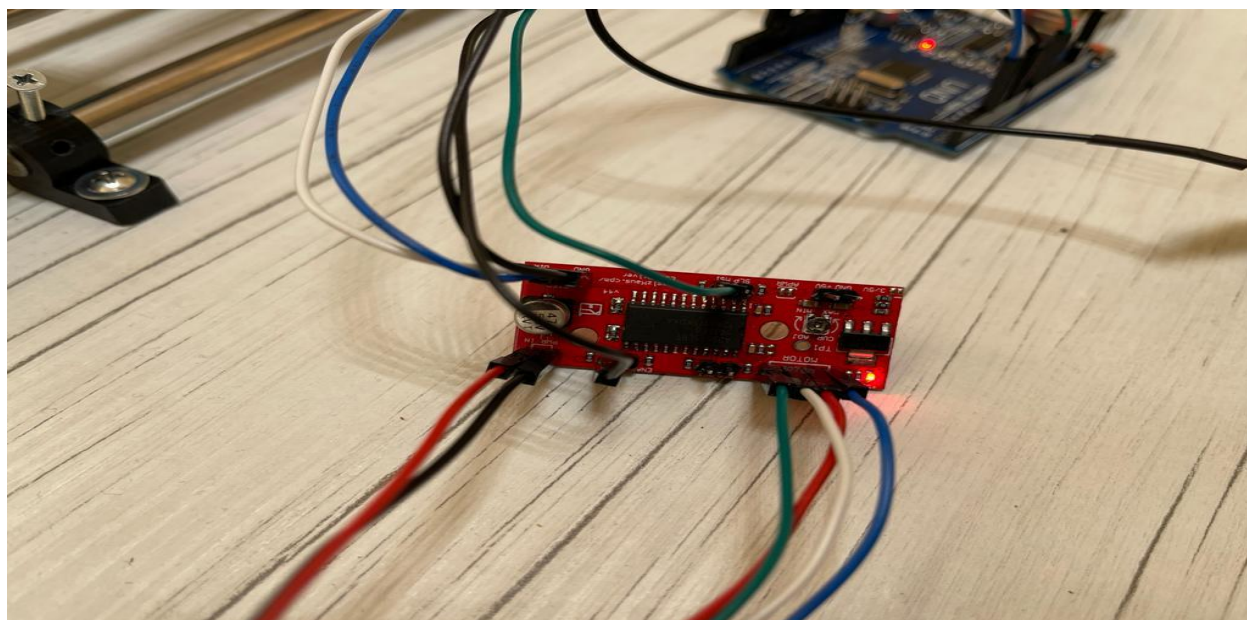


Descriere:

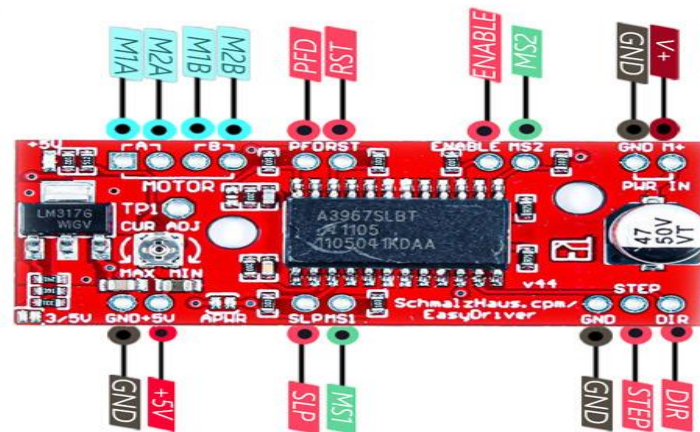
Un motor pas cu pas este un tip de motor electric care își deplasează rotorul în pași individuali, cu un control precis al unghiului de rotație. Acest lucru permite poziționarea exactă și mișcarea repetitivă fără a avea nevoie de senzori de poziție externi.

Fiecare rotație completă a unui motor pas cu pas este împărțită într-un număr fix de pași. Controlerul motorului trimite semnale de comandă, care sunt impulsuri electrice, și fiecare impuls determină motorul să se miște un pas. Prin urmare, se poate controla cu precizie cât de mult se rotește motorul prin numărul de impulsuri trimise.

Motoarele pas cu pas sunt utilizate frecvent în aplicații care necesită control precis al mișcării, cum ar fi imprimantele 3D, mașinile de gravat cu laser, camerele de poziționare și în echipamentele de automatizare. Avantajele lor includ precizia, repetabilitatea, și capacitatea de a fi controlate ușor cu circuite electronice și microcontrolere, cum ar fi Arduino.



MOTOR VOLTAGE
 GND
 LOGIC VOLTAGE
 MOTOR PINS
 CONTROL PINS
 STEP RESOLUTIONS

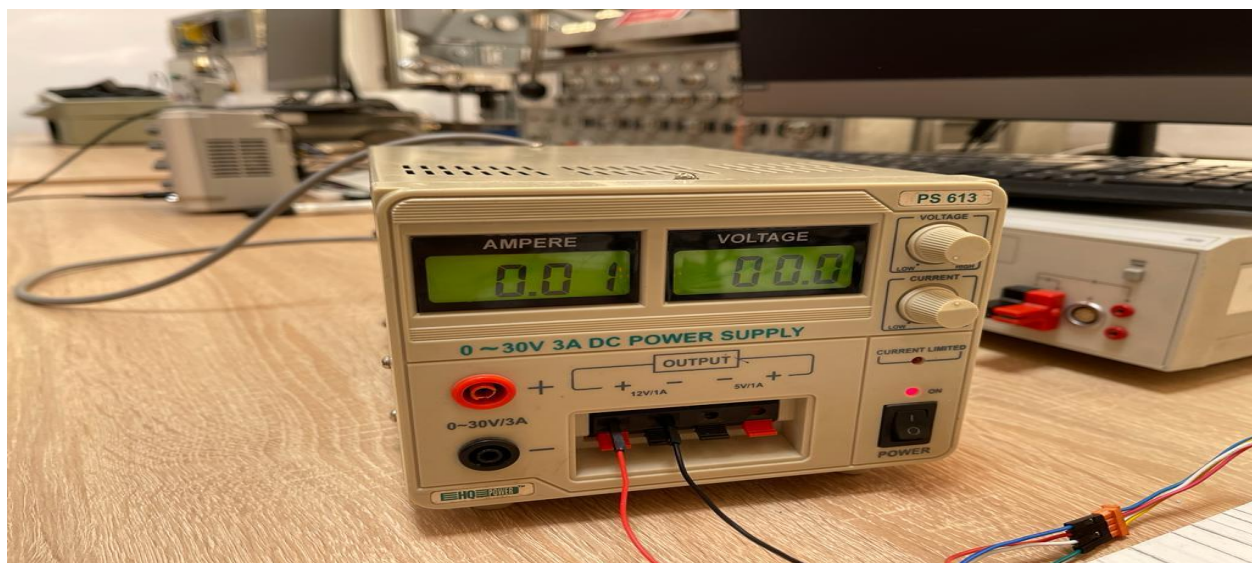


www.Electropeak.com



MS ₁	MS ₂	Resolution
L	L	Full step (2 phase)
H	L	Half step
L	H	Quarter step
H	H	Eighth step

Motorul pas cu pas functioneaza cel mai bine la o tensiune de 12 V.



Motor pas cu pas	Pasul = 2700	2700 = 1cm
------------------	--------------	------------

(2)Partea Software

- 1) Codul folosit pe Arduino Uno pentru a putea masura capacitatea
- 2) Interfata creata in Python – aceasta este formata din doua parti. O parte este formata dintr-un cod care afiseaza valorile masurate de la arduino si celalat cod care afiseaza un grafic cu valorile distantei si capacitatii
- 3) Grafic creat in LabView;

1) Cod Arduino(doar pentru masurarea capacitatii):

```
const int OUT_PIN = A2;
const int IN_PIN = A0;

//Capacitance between IN_PIN and Ground
//Stray capacitance is always present. Extra capacitance can be added to
//allow higher capacitance to be measured.
const float IN_STRAY_CAP_TO_GND = 24.48; //initially this was 30.00
const float IN_EXTRA_CAP_TO_GND = 0.0;
const float IN_CAP_TO_GND = IN_STRAY_CAP_TO_GND + IN_EXTRA_CAP_TO_GND;
const int MAX_ADC_VALUE = 1023;

void setup()
{
  pinMode(OUT_PIN, OUTPUT);
  //digitalWrite(OUT_PIN, LOW); //This is the default state for outputs
  pinMode(IN_PIN, OUTPUT);
  //digitalWrite(IN_PIN, LOW);

  Serial.begin(115200);
}

void loop()
{
  //Capacitor under test between OUT_PIN and IN_PIN
  //Rising high edge on OUT_PIN
  pinMode(IN_PIN, INPUT);
  digitalWrite(OUT_PIN, HIGH);
  int val = analogRead(IN_PIN);

  //Clear everything for next measurement
  digitalWrite(OUT_PIN, LOW);
  pinMode(IN_PIN, OUTPUT);
}
```

```
//Calculate and print result

float capacitance = (float)val * IN_CAP_TO_GND / (float)(MAX_ADC_VALUE - val);

Serial.print(F("Capacitance Value = "));
Serial.print(capitance, 3);
Serial.print(F(" pF ("));
Serial.print(val);
Serial.println(F(")"));

while (millis() % 500 != 0)
;
}
```

Acest cod este folosit pentru a măsura capacitatea unui condensator care este conectat între pini definiți ca OUT_PIN și IN_PIN.

Aici este o explicație detaliată a fiecărui segment al codului:

- 1) Inițializează constantele pentru pini și capacitățile implicite:
 - OUT_PIN este setat la A2, care este pinul de ieșire către condensator.
 - IN_PIN este setat la A0, care este pinul de intrare de la condensator.
 - IN_STRAY_CAP_TO_GND este capacitatea parazită a pinului de intrare față de pământ. Aceasta este valoarea capacității care este mereu prezentă, chiar dacă nu este conectat un condensator extern.
 - IN_EXTRA_CAP_TO_GND permite adăugarea unei capacități suplimentare pentru a măsura capacități mai mari. În acest caz, este setat la 0.0.
 - IN_CAP_TO_GND este suma capacității parazite și a capacității suplimentare.

- MAX_ADC_VALUE este valoarea maximă pe care ADC-ul (Analog Digital Converter) o poate citi, 1023 pentru un Arduino tipic cu un ADC de 10 biți.

2) Funcția setup() este apelată o singură dată când programul pornește:

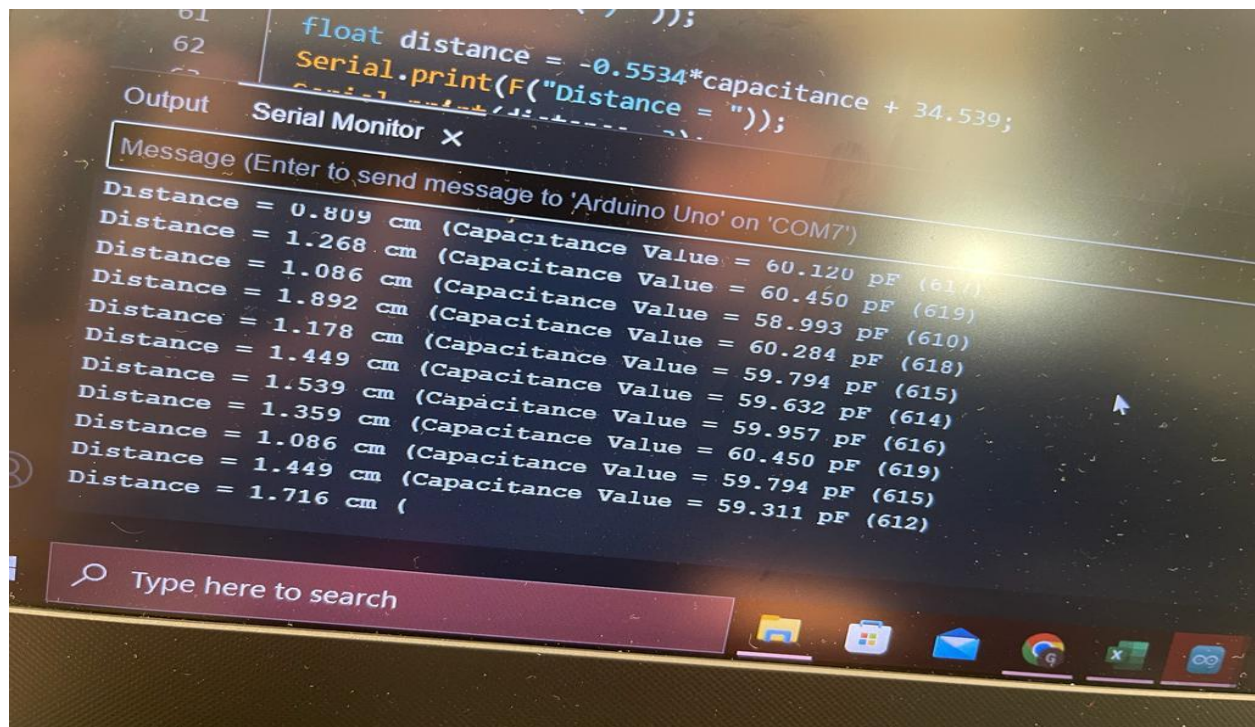
- Setează OUT_PIN ca pin de ieșire.
- Inițializează comunicația Serială la 115200 baud pentru a transmite date la calculator.

3) Funcția loop() se repetă la infinit și este inima măsurătorii capacității:

- Setează IN_PIN ca pin de intrare.
- Pune OUT_PIN în stare HIGH (5V), ceea ce începe încărcarea condensatorului.
- Citește valoarea tensiunii pe IN_PIN folosind analogRead(), care este proporțională cu capacitatea condensatorului.
- Pregătește totul pentru următoarea măsurătoare prin setarea OUT_PIN în LOW și IN_PIN înapoi ca pin de ieșire.
- Calculează capacitatea folosind valoarea citită și constantele definite, apoi o afișează prin portul serial.
- La sfârșitul buclei, există o întârziere până când timpul sistemului este un multiplu de 500 de milisecunde, aceasta pentru a limita cât de des se execută bucla.

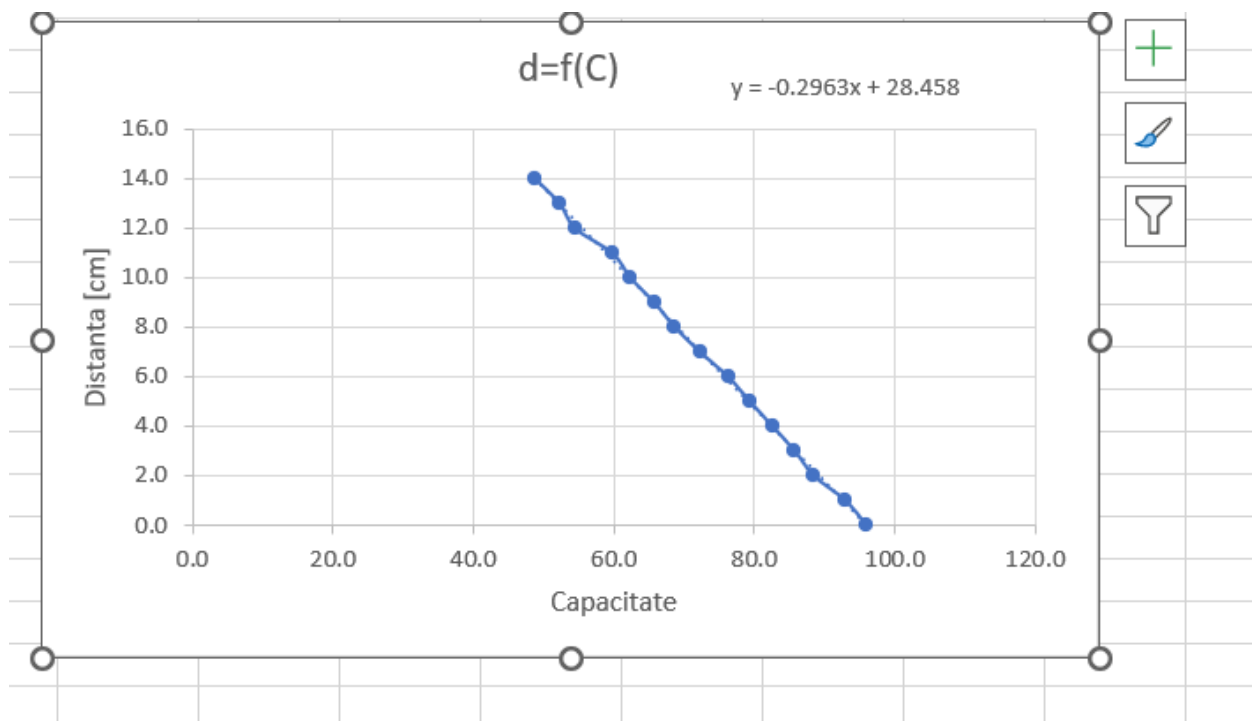
Capacitatea calculată este afișată în picofarazi (pF) și este rezultatul unei simplificări a formulei pentru un circuit RC, unde val este tensiunea pe condensator în momentul citirii. Această abordare este utilizată când se

măsoară o capacitate mică, iar condițiile de mediu și alte influențe externe sunt relativ constante și cunoscute.



Masurari :

Distanta(cm)	Capacitate(pF)
0.0	96.0
1.0	93.0
2.0	88.5
3.0	85.8
4.0	82.6
5.0	79.4
6.0	76.3
7.0	72.3
8.0	68.7
9.0	65.8
10.0	62.4
11.0	59.8
12.0	54.5
13.0	52.3
14.0	48.8



Aparatul de masura

2) Cod Python(afisarea valorilor de la ARDUINO si graficul):

```
# matplotlib, pyside6, serial, pyqtgraph
from itertools import repeat

import serial
import re
import time
import sys
import random
from pyqtgraph import PlotWidget
from collections import deque
from PyQt6 import QtWidgets, QtCore
from PySide6.QtWidgets import QVBoxLayout
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as
FigureCanvas
from matplotlib.figure import Figure

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
                             QMetaObject, QObject, QPoint, QRect,
                             QSize, QTime, QUrl, Qt, QTimer)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
                             QFont, QFontDatabase, QGradient, QIcon,
                             QImage, QKeySequence, QLinearGradient, QPainter,
                             QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QHBoxLayout, QLabel, QLineEdit,
                                QMainWindow, QMenuBar, QSizePolicy,
                                QStatusBar,
                                QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 746)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.widget = QWidget(self.centralwidget)
        self.widget.setObjectName(u"widget")
        self.widget.setGeometry(QRect(0, 160, 891, 290))
        self.verticalLayout = QVBoxLayout(self.widget)
        self.verticalLayout.setObjectName(u"verticalLayout")
        self.verticalLayout.setContentsMargins(0, 0, 0, 0)
        self.horizontalLayout_3 = QHBoxLayout()
        self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
        self.label = QLabel(self.widget)
        self.label.setObjectName(u"label")
        font = QFont()
        font.setPointSize(48)
        self.label.setFont(font)
```

```
self.horizontalLayout_3.addWidget(self.label)

self.lineEdit_capacitate = QLineEdit(self.widget)
self.lineEdit_capacitate.setObjectName(u"lineEdit_capacitate")
self.lineEdit_capacitate.setFont(font)
self.lineEdit_capacitate.setReadOnly(True)

self.horizontalLayout_3.addWidget(self.lineEdit_capacitate)

self.verticalLayout.addLayout(self.horizontalLayout_3)

self.horizontalLayout_2 = QHBoxLayout()
self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
self.label_2 = QLabel(self.widget)
self.label_2.setObjectName(u"label_2")
self.label_2.setFont(font)

self.horizontalLayout_2.addWidget(self.label_2)

self.lineEdit_distanta = QLineEdit(self.widget)
self.lineEdit_distanta.setObjectName(u"lineEdit_distanta")
self.lineEdit_distanta.setFont(font)
self.lineEdit_distanta.setReadOnly(True)

self.horizontalLayout_2.addWidget(self.lineEdit_distanta)

self.verticalLayout.addLayout(self.horizontalLayout_2)

self.horizontalLayout = QHBoxLayout()
self.horizontalLayout.setObjectName(u"horizontalLayout")
self.label_3 = QLabel(self.widget)
self.label_3.setObjectName(u"label_3")
self.label_3.setFont(font)

self.horizontalLayout.addWidget(self.label_3)

self.lineEdit_distanta_ir = QLineEdit(self.widget)
self.lineEdit_distanta_ir.setObjectName(u"lineEdit_distanta_ir")
self.lineEdit_distanta_ir.setFont(font)
self.lineEdit_distanta_ir.setReadOnly(True)

self.horizontalLayout.addWidget(self.lineEdit_distanta_ir)

self.verticalLayout.addLayout(self.horizontalLayout)

MainWindow.setCentralWidget(self.centralwidget)
self.menubar = QMenuBar(MainWindow)
self.menubar.setObjectName(u"menubar")
```

```

        self.menubar.setGeometry(QRect(0, 0, 800, 22))
        MainWindow.setMenuBar(self.menubar)
        self.statusbar = QStatusBar(MainWindow)
        self.statusbar.setObjectName(u"statusbar")
        MainWindow.setStatusBar(self.statusbar)

        self.retranslateUi(MainWindow)

        QMetaObject.connectSlotsByName(MainWindow)
# setupUi

    def retranslateUi(self, MainWindow):
        MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"MainWindow", None))
        self.label.setText(QCoreApplication.translate("MainWindow",
u"Capacitate: ", None))
        self.label_2.setText(QCoreApplication.translate("MainWindow",
u"Distanța: ", None))
        self.label_3.setText(QCoreApplication.translate("MainWindow",
u"Distanța(IR): ", None))
# retranslateUi

class MainWindow(QMainWindow, Ui_MainWindow):
    def __init__(self):
        super(MainWindow, self).__init__()
        self.setupUi(self)
        self.initUI()

    def initUI(self):
        self.arduino = serial.Serial('COM3', 115200) # Înlocuiește 'COMx' cu
portul serial al Arduino

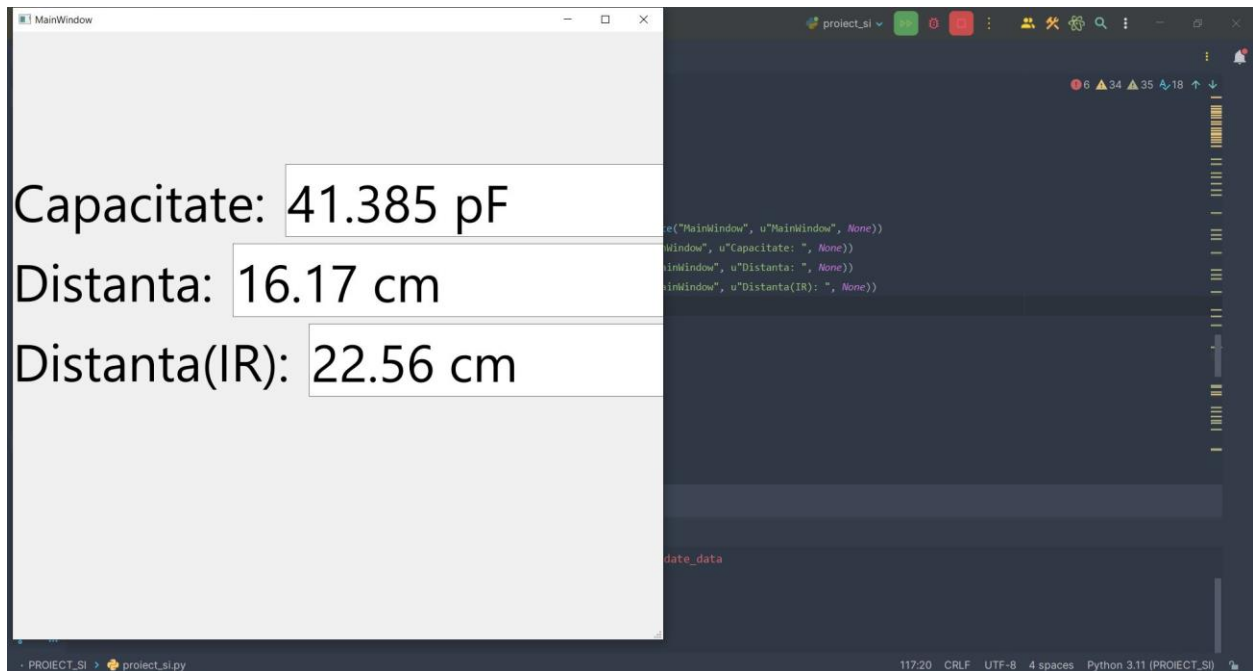
        self.timer = QTimer(self)
        self.timer.timeout.connect(self.update_data)
        self.timer.start(100) # Actualizează datele la fiecare 100 ms

    def update_data(self):
        data = self.arduino.readline().decode('utf-8').rstrip()
        capacitance = float(re.search("(?<=Value = )[0-9]+\.[0-
9]+",data).group(0))
        self.lineEdit_capacitate.setText(str(capacitance) + " pF")
        distance = float(re.search("(?<=Distance = )[0-9]+\.[0-
9]+",data).group(0))
        self.lineEdit_distanța.setText(str(distance) + " cm")
        distance_2 = float(re.search("(?<=IR = )[0-9]+\.[0-
9]+",data).group(0))
        self.lineEdit_distanța_ir.setText(str(distance_2)+ " cm")

```

```
def main():
    app = QApplication()
    window = MainWindow()
    window.show()
    app.exec()

if __name__ == "__main__":
    main()
```



```
# python .\SI.py --port COM3

import sys, serial, argparse
import numpy as np
from time import sleep
from collections import deque

import matplotlib.pyplot as plt
import matplotlib.animation as animation

# plot class
class AnalogPlot:
    # constr
    def __init__(self, strPort, maxLen):
        # open serial port
        self.ser = serial.Serial(strPort, 115200)

        self.ax = deque([0.0]*maxLen)
```

```

        self.ay = deque([0.0]*maxLen)
        self.maxLen = maxLen

# add to buffer
def addToBuf(self, buf, val):
    if len(buf) < self.maxLen:
        buf.append(val)
    else:
        buf.pop()
        buf.appendleft(val)

# add data
def add(self, data):
    assert(len(data) == 2)
    self.addToBuf(self.ay, data[0])
    self.addToBuf(self.ax, data[1])

# update plot
def update(self, frameNum, a0, a1):
    try:
        line = self.ser.readline()
        data = [float(val) for val in line.split()]
        # print data
        if(len(data) == 2):
            self.add(data)
            a0.set_data(range(self.maxLen), self.ax)
            a1.set_data(range(self.maxLen), self.ay)
            text.set_text(f'Rez: {data[0]:.2f}\nLux: {data[1]:.2f}\nLED:
{data[2]:.2f}')
        except KeyboardInterrupt:
            print('exiting')

    return a0,

# clean up
def close(self):
    # close serial
    self.ser.flush()
    self.ser.close()

# main() function
def main():
    # create parser
    parser = argparse.ArgumentParser(description="LDR serial")
    # add expected arguments
    parser.add_argument('--port', dest='port', required=True)

    # parse args
    args = parser.parse_args()

    strPort = '/dev/tty.usbserial-A7006Yqh'
    strPort = args.port

```

```

print('reading from serial port %s...' % strPort)

# plot parameters
analogPlot = AnalogPlot(strPort, 100)

print('plotting data...')

# set up animation
fig = plt.figure()
ax = plt.axes(xlim=(0, 20), ylim=(0, 100))
a0, = ax.plot([], [], label = "Distanta")
a1, = ax.plot([], [], label = "Capacitate")
ax.legend()
anim = animation.FuncAnimation(fig, analogPlot.update,
                               fargs=(a0, a1),
                               interval=50)

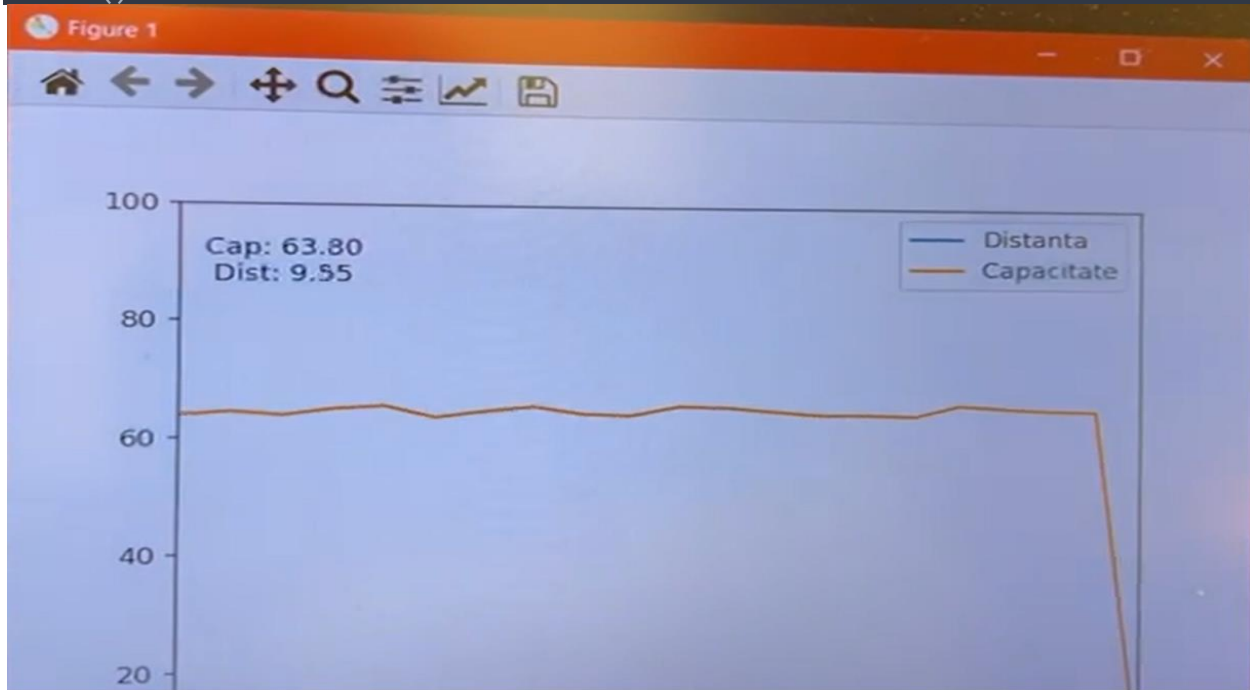
# show plot
plt.show()

# clean up
analogPlot.close()

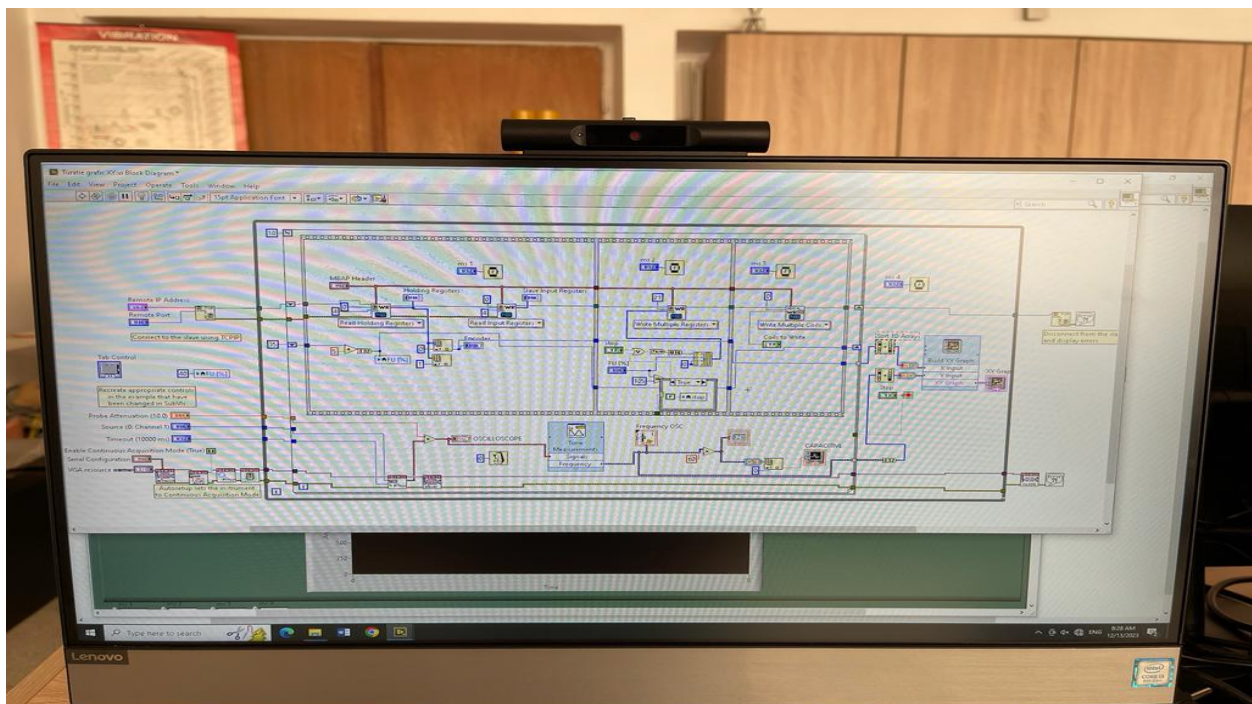
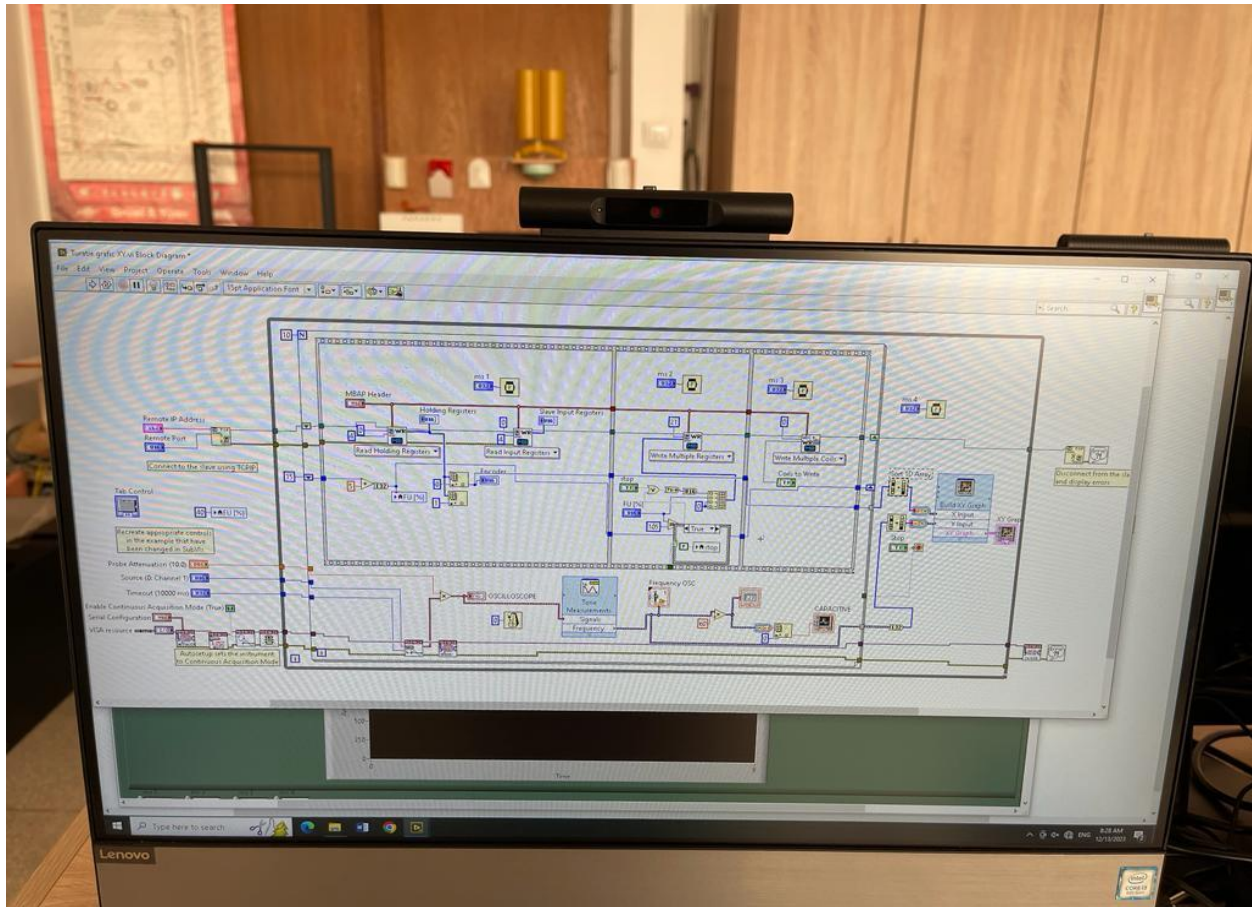
print('exiting.')

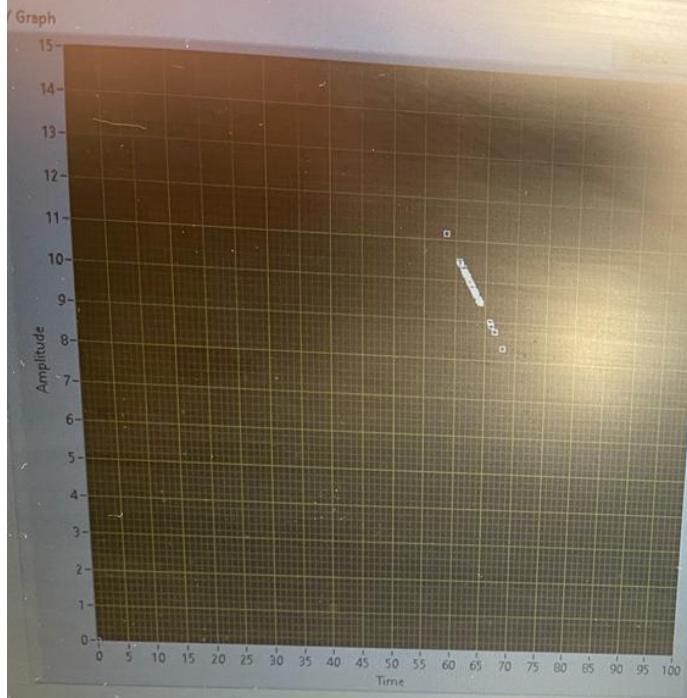
# call main
if __name__ == '__main__':
    main()

```



3) Labview :





number number 2 substring

61.6999 10.1791 61.699934

number 3 substring 2

10.1793 10.179274

concatenated string

60.693001 10.474665
62.260429 10.010235
61.131428 10.344759
62.843296 9.837532
60.911297 10.409983
63.438950 9.661039
61.465118 10.245886
62.145355 10.044333
60.693001 10.474665
63.802650 9.553276
62.608639 9.907061
62.843296 9.837532
61.689934 10.179274
62.375999 9.975992
63.680882 9.589355
63.438950 9.661039
61.465118 10.245886
62.961388 9.802542
60.911297 10.409983

appended array

10x
0
0
0
0
0
0
0
0
0
0

appended array 2

0
0
0
0
0
0
0
0
0
0
0

Cod integral pentru ARDUINO UNO (masoara capacitatea si pune in functiune motorului pas cu pas) :

```
#define stp 2
#define dir 3
#define MS1 4
#define MS2 5

int cal_cap = 33;
int cal_dist = 0;
int dist_motor = 1;
char resultBuffer[128];
char capacitanceStr[32];
char distanceStr[32];

int x;
int nr=0;

#include <SPI.h>
#include <Wire.h>
const int OUT_PIN = A2;
const int IN_PIN = A0;

//Capacitance between IN_PIN and Ground
//Stray capacitance is always present. Extra capacitance can be added to
//allow higher capacitance to be measured.
const float IN_STRAY_CAP_TO_GND = 24.48; //initially this was 30.00, then 24.48 (for seeeduino lite 26.40)
const float IN_EXTRA_CAP_TO_GND = 0.0;
const float IN_CAP_TO_GND = IN_STRAY_CAP_TO_GND + IN_EXTRA_CAP_TO_GND;
const int MAX_ADC_VALUE = 1023;

void setup()
{
  pinMode(OUT_PIN, OUTPUT);
  //digitalWrite(OUT_PIN, LOW); //This is the default state for outputs
  pinMode(IN_PIN, OUTPUT);
  //digitalWrite(IN_PIN, LOW);

  // by default, we'll generate the high voltage from the 3.3v line internally! (neat!)
  //display.begin(SSD1306_SWITCHCAPVCC, 0x3C); // initialize with the I2C addr 0x3D (for the 128x64)
  // init done

  // Clear the buffer.
  //display.clearDisplay();
  // Set text size and color
  //display.setTextColor(WHITE);

  pinMode(stp, OUTPUT);
  pinMode(dir, OUTPUT);
  pinMode(MS1, OUTPUT);
  pinMode(MS2, OUTPUT);
```

```

digitalWrite(MS1,LOW);
digitalWrite(MS2,LOW);

Serial.begin(115200);
}

void loop()
{
  //Capacitor under test between OUT_PIN and IN_PIN
  //Rising high edge on OUT_PIN
  pinMode(IN_PIN, INPUT);
  digitalWrite(OUT_PIN, HIGH);
  int val = analogRead(IN_PIN);

  //Clear everything for next measurement
  digitalWrite(OUT_PIN, LOW);
  pinMode(IN_PIN, OUTPUT);

  //Calculate and print result

  float capacitance = (float)val * IN_CAP_TO_GND / (float)(MAX_ADC_VALUE - val) + cal_cap ;

  Serial.print(F("Capacitance Value = "));
  Serial.print(capacitance, 3);
  Serial.print(F(" pF ("));
  Serial.print(val);
  Serial.println(F(")"));

  float distance = -0.2963 *capacitance + 28.458 + cal_dist;

  Serial.print(F("Distance = "));
  Serial.print(distance, 3);
  Serial.print(F(" cm ("));

  digitalWrite(dir, HIGH);
  while(nr<1)
  {
    for(x= 0; x< dist_motor * 2700; x++)
    {
      digitalWrite(stp,HIGH);
      delay(1);
      digitalWrite(stp,LOW);
      delay(1);
    }
    nr++;
  }

  /*-----PLOT DISTANTA SI CAPACITATE*/
  /*dtostrf(distance, 8, 6, distanceStr);
  dtostrf(capacitance, 8, 6, capacitanceStr);

```

```
    sprintf(resultBuffer, "%s %s", capacitanceStr, distanceStr);  
    Serial.println(resultBuffer);  
    */  
    while (millis() % 500 != 0);  
}
```

Experimente pentru verificarea functionalitatii :

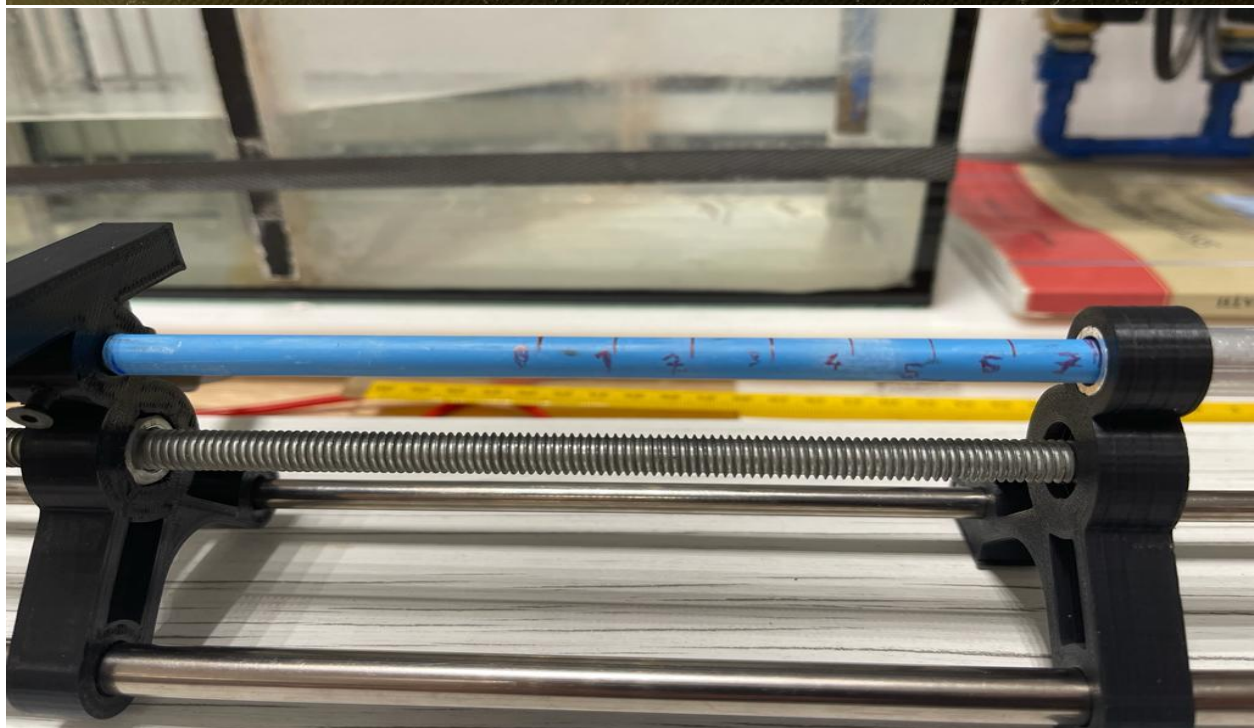
```
93   nr++;
94   }
95   delay(1000);
```

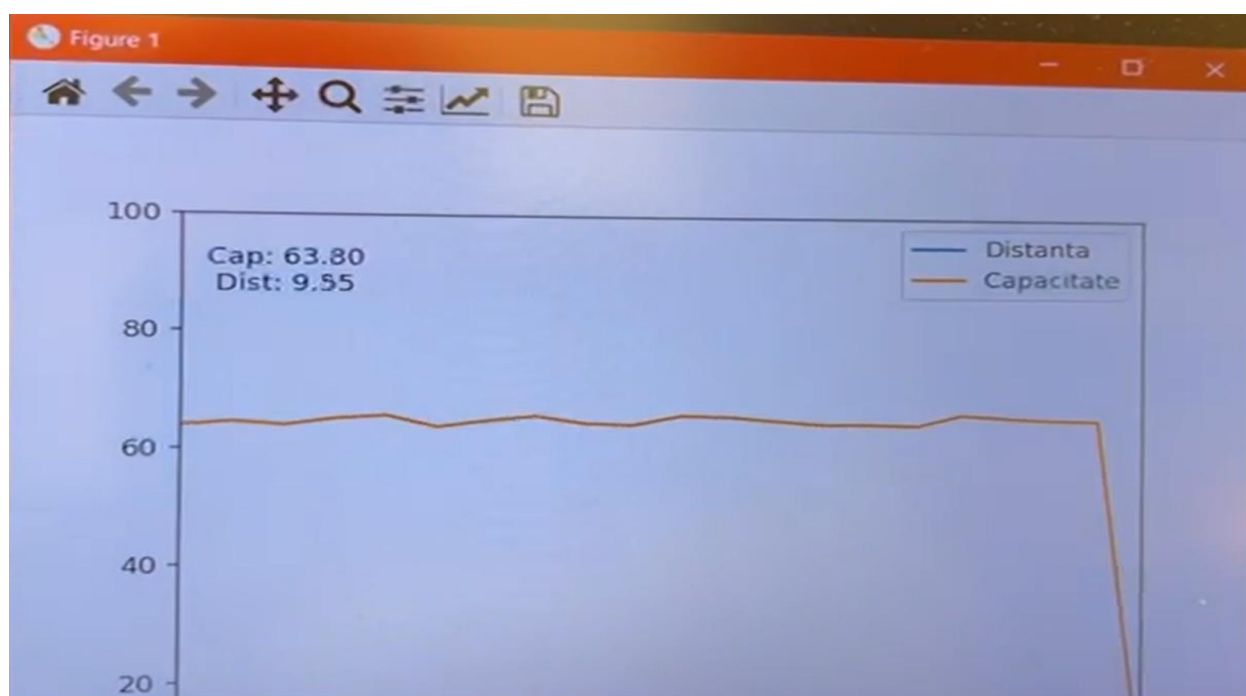
Output Serial Monitor ✕

Message (Enter to send message to 'Arduino Uno' on 'COM10')

Distance = 1.158 cm (Capacitance Value = 11.966 pF (518))
Distance = 7.134 cm (Capacitance Value = 69.863 pF (564))
Distance = 7.758 cm (Capacitance Value = 71.812 pF (577))
Distance = 7.180 cm (Capacitance Value = 69.000 pF (558))
Distance = 8.013 cm (Capacitance Value = 71.658 pF (576))
Distance = 7.226 cm (Capacitance Value = 71.812 pF (577))
Distance = 7.180 cm (Capacitance Value = 71.812 pF (577))
Distance = 7.180 cm (Capacitance Value = 72.750 pF (583))
Distance = 6.902 cm (Capacitance Value = 70.155 pF (566))
Distance = 7.671 cm (Capacitance Value = 72.122 pF (579))
Distance = 7.088 cm (

Type here to search





OBSERVATII :

În esență, performanța proiectului este satisfăcătoare, cu mențiunea că precizia este garantată doar în intervalul de 6-10 cm. Dincolo de acest domeniu, formula de calcul derivată din măsurătorile efectuate cu puntea de măsură devine imprecisă.

Această imprecizie survine din cauza capacităților parazite care apar în prezența cablurilor utilizate în actuala configurație. Astfel de capacități nedorite afectează acuratețea măsurătorilor.

Pentru a îmbunătăți precizia, ar fi indicat să se utilizeze un senzor capacitiv diferențial, care este mai rezistent la interferențele externe și ar trebui să aibă o precizie mai bună. În plus, ar trebui redus la minim posibil utilizarea cablurilor, pentru a diminua influența capacităților parazite asupra măsurătorilor.

Bibliografie:

- ✓ <https://chat.openai.com/>
- ✓ <https://www.circuito.io/blog/arduino-uno-pinout/>
- ✓ https://www.researchgate.net/figure/Electric-diagram-of-the-capacitive-sensor-circuit-The-proposed-model-consists-of-the_fig9_333827263
- ✓ Curs Senzori Inteligenti
- ✓ Curs Senzori si Traductoare
- ✓ <https://electropeak.com/learn/interfacing-a3967-easydriver-stepper-motor-driver-with-arduino/>
- ✓ <https://wordpress.codewrite.co.uk/pic/2014/01/21/cap-meter-with-arduino-uno/>
- ✓ <https://www.tme.eu/ro/news/library-articles/page/41861/Motorul-pas-cu-pas-in-salturi-tipuri-i-exemple-de-utilizare-a-motoarelor-pas-cu-pas/>
- ✓ https://en.wikipedia.org/wiki/Arduino_Uno